NOP LOAD1: MAR ⟵ PC
LOADI Rn,X 0001000nX Rn ⟵ X
IF1: MAR ⟵ PC
IF2: MDR ⟵ M[MAR], PC <- PC + 1
IF3: IR ⟵ MDR

LOADI1: MAR ⟵ PC
LOADI2: MDR ⟵ M[MAR], PC ⟵ PC+1
LOADI3: R0 ⟵ MDR if IR(0) = 0, R1 ⟵ MDR if IR(0) = 1

LOAD Rn,X 0010000n X Rn ⟵ M[X]
IF1: MAR ⟵ PC
IF2: MDR ⟵ M[MAR], PC <- PC + 1
IF3: IR ⟵ MDR

LOAD1: MAR ⟵ PC
LOAD2: MDR ⟵ M[MAR], PC ⟵ PC+1
LOAD3: MAR ⟵ MDR
LOAD4: MDR ⟵ M[MAR]
LOAD5: R0 ⟵ MDR if IR(0) = 0, R1 ⟵ MDR if IR(0) = 1

STORE X,Rm 0011000m M[X] ⟵ Rm
IF1: MAR ⟵ PC
IF2: MDR ⟵ M[MAR], PC <- PC + 1
IF3: IR ⟵ MDR

STORE1: MAR ⟵ PC
STORE2: MDR ⟵ M[MAR], PC ⟵ PC+1
STORE3: MAR ⟵ MDR
STORE4: MDR ⟵ R0 if IR(0) = 0, MDR ⟵ R1 if IR(0) = 1
STORE5: M[MAR] ⟵ MDR

MOVE Rn,Rm 0100000n Rn ⟵ Rm, m ≠ n
IF1: MAR ⟵ PC
IF2: MDR ⟵ M[MAR], PC <- PC + 1
IF3: IR ⟵ MDR

MOVE1: R0 ⟵ R1 if IR(0) = 0, R1 ⟵ R0 if IR(0) = 1

ADD Rn,Rm 0101000n Rn ⟵ Rn + Rm, m ≠ n
IF1: MAR ⟵ PC
IF2: MDR ⟵ M[MAR], PC <- PC + 1
IF3: IR ⟵ MDR

ADD1: R0 ⟵ R0 + R1 if IR(0) = 0, R1 ⟵ R1 + R0 if IR(0) = 1

SUB Rn,Rm 0110000n Rn ⟵ Rn – Rm, m ≠ n
IF1: MAR ⟵ PC
IF2: MDR ⟵ M[MAR], PC <- PC + 1
IF3: IR ⟵ MDR

SUB1: R0 ⟵ R0 + R1’ + 1 if IR(0) = 0, R1 ⟵ R1 + R0’ + 1 if IR(0) = 1

TESTNZ Rm 0111000m Z ⟵ not V, V = OR of the bits of Rm
--test for not zero flag
IF1: MAR <- PC
IF2: MDR <- M[MAR], PC <- PC + 1
IF3: IR <- MDR

TESTNZ1: X <- or(R0) if IR(0) = 0, X <- or(R1) if IR(0) = 1
TESTNZ2: Z <- not X

TESTZ Rm 1000000m Z ⟵ V, V = OR of the bits of Rm
--test for zero flag
IF1: MAR <- PC
IF2: MDR <- M[MAR], PC <- PC + 1
IF3: IR <- MDR

TESTZ1: X <- or(R0) if IR(0) = 0, X <- or(R1) if IR(0) = 1
TESTZ2: Z <- X

JUMP X 10010000 X PC ⟵ X
IF1: MAR <- PC
IF2: MDR <- M[MAR], PC <- PC + 1
IF3: IR <- MDR

JUMP1: MAR ⟵ PC
JUMP2: MDR ⟵ M[MAR], PC ⟵ PC+1
JUMP3: PC ⟵ MDR

JUMPZ X 10100000 X If (Z = 1) then PC ⟵ X
IF1: MAR <- PC
IF2: MDR <- M[MAR], PC <- PC + 1
IF3: IR <- MDR

JUMPZ1: MAR ⟵ PC
JUMPZ2: MDR ⟵ M[MAR], PC ⟵ PC+1

JUMPZ3: if (Z = 1) then PC ⟵ MDR

LOADSP X   10110000   X SP ⟵ X
IF1: MAR <- PC
IF2: MDR <- M[MAR], PC <- PC + 1
IF3: IR <- MDR

LOADSP1: MAR ⟵ PC
LOADSP2: MDR ⟵ M[MAR], PC ⟵ PC+1
LOADSP3: SP ⟵MDR

PEEP Rn   1100000n   Rn ⟵ M[SP]
IF1: MAR <- PC
IF2: MDR <- M[MAR], PC <- PC + 1
IF3: IR <- MDR

PEEP1: MAR ⟵ SP
PEEP2: MDR ⟵ M[MAR]
PEEP3: R0 ⟵ MDR if IR(0) = 0, R1 ⟵ MDR if IR(0) = 1

PUSH Rn   1101000n   M[--SP] ⟵ Rn Pre-increment
IF1: MAR <- PC
IF2: MDR <- M[MAR], PC <- PC + 1
IF3: IR <- MDR

PUSH1: SP ⟵ SP - 1
PUSH2: MAR ⟵ SP
PUSH3: MDR ⟵ R0 if IR(0) = 0, MDR ⟵ R1 if IR(0) = 1
PUSH4: M[MAR] ⟵ MDR

POP Rn   1110000n   Rn ⟵ M[SP++] Post-decrement
IF1: MAR ⟵ PC
IF2: MDR ⟵ M[MAR], PC <- PC + 1
IF3: IR ⟵ MDR

POP1: MAR ⟵ SP
POP2: MDR ⟵ M[MAR]
POP3: SP ⟵ SP+1
POP4: R0 ⟵ MDR if IR(0) = 0, R1 ⟵ MDR if IR(0) = 1

HALT   11110000   PC ⟵ 0, stop microsequencer
IF1: MAR ⟵ PC
IF2: MDR ⟵ M[MAR], PC <- PC + 1
IF3: IR ⟵ MDR

HALT: PC $\leftarrow$ 0

X = 8-bit data or address; m, n = 0 or 1, m $\neq$ n

Program Counter (PC) – 8 bits
Memory Address Register (MAR) – 8 bits
Instruction Register (IR) – 8 bits
Memory Data Register (MDR) – 8 bits
Register File (R0-R1) – 8 bits
Condition flip-flop (Z) – 1 bit $\leftarrow$ part of the enable/load of registers?
Stack Pointer (SP) – 8 bits